

Course Outline: Understanding Telemetry from the Frontend

Title: Understanding Telemetry from the Frontend **Skill:** Skill 43 — Collecting telemetry and context-related info from the user and your application **Learnpack:** + Entendiendo la telemetría desde el frontend **File:** s43-w14d42-entendiendo-la-telemetria-desde-el-frontend **Prerequisite:** Telemetry Architecture (s42-w14d41) **Target Audience:** Beginner developers building frontend applications who want to understand what data to collect and why **Total Lessons:** 8 **Format:** Conceptual (0% hands-on)

Course Description

Your application is running. Users are clicking, navigating, and occasionally running into problems — and you have no idea what they're experiencing. This course answers the fundamental questions of frontend telemetry: what data should you collect from your users and application, why does each category matter, and how do you think about the distinction between a user's profile and their behavioral data? By the end, you will have a clear mental model for designing a telemetry collection strategy from scratch.

Course Philosophy

This course emphasizes:

- Intentionality: collecting only what enables a specific decision
 - Privacy by default: evaluating the sensitivity of data before collecting it
 - The minimum useful set: capturing enough to understand "what happened" without interrogating the user or reading source code
 - Analytical quality: defining events that measure real value (funnel, activation, retention)
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - What is Telemetry	1
02 - What to Collect	3
03 - User Profile vs Usage Data	1
04 - Assessment	1
05 - Outro	1
Total	8

00.0 Welcome to Frontend Telemetry 📖

Learning Objectives:

- Understand why frontend telemetry has become essential for modern product development
- Preview the four questions this course will answer: what telemetry is, what to collect, why to collect it, and how to distinguish user state from user behavior

Content Outline:

- The gap: your application runs silently — users encounter bugs, confusion, and friction, but you only find out when they complain (or leave)
- Telemetry as the solution: systematic, automatic collection of data about what your application and its users are actually doing
- The four things this course will teach:
 1. What telemetry is and what problem it solves
 2. The seven categories of data worth collecting
 3. The five objectives that justify collecting each category
 4. The critical distinction between user profile data and usage event data
- What you will be able to do by the end: design a telemetry collection strategy that is intentional, privacy-respecting, and actionable

Transitions: This lesson frames the problem. Lesson 01.0 dives into what telemetry actually is and why it matters more now than ever.

Key Principles:

- Telemetry without intention is surveillance — you need to know why you're collecting before you collect
- The minimum useful set: collect enough to understand what happened, no more

Examples:

- A product team learns that 40% of users abandon the checkout at step 3 — only possible because telemetry captured funnel completion rates
 - A developer gets a bug report: "it doesn't work" — with no telemetry, reproduction is impossible; with telemetry, the session context, error fingerprint, and navigation breadcrumbs make it trivial
-

01.0 What is Telemetry 📖

Learning Objectives:

- Define telemetry in the context of frontend applications
- Explain what problem telemetry solves that logging alone cannot
- Distinguish between reactive debugging (responding to complaints) and proactive observability (detecting problems before users report them)

Content Outline:

- What telemetry is: the automatic collection of measurements and data from a running system, sent to a backend for analysis
- What it is NOT: traditional server-side logging, manual bug reporting, or user surveys
- The three things telemetry enables that you cannot do without it:
 1. Observability: knowing the health of your system in real time
 2. Product analytics: understanding how users actually use your application (not how you think they use it)
 3. Reproducibility: being able to reconstruct exactly what a user experienced when a bug occurred
- Why frontend telemetry specifically: the browser is a hostile, unpredictable environment — different devices, network conditions, browser versions, and user behaviors create problems that never appear in development
- The planning principle: telemetry must be designed at the start of development, not bolted on after the first incident

Transitions: Now that you understand what telemetry is and what it enables, the next question is: what data should you actually collect? Section 02 maps out the seven categories.

Key Principles:

- Telemetry converts invisible user experiences into observable, measurable data
- Reactive debugging (waiting for complaints) scales poorly; proactive telemetry lets you detect and fix problems before users notice
- Plan telemetry at the start — retrofitting it later is expensive and incomplete

Examples:

- Without telemetry: a user reports "the app is slow." You cannot reproduce it, don't know their device, network, or which route triggered it
- With telemetry: you see that 8% of mobile users on 3G experience API latency above 2 seconds on `/checkout` — actionable, reproducible, quantified
- Anti-pattern: treating telemetry as a DevOps concern added post-launch — the product team never gets the data they need, and the developer can't reproduce user-reported bugs

02.0 What to Collect

Learning Objectives:

- Understand that telemetry data falls into distinct categories, each serving a different purpose
- Apply the "minimum useful set" principle to avoid over-collection
- See how the seven categories together create a complete picture of user context

Content Outline:

- The fundamental question before collecting any data: "What decision does this data enable?"
- Why categories matter: unstructured collection leads to data dumps that nobody can analyze
- The seven categories overview:
 1. Identity / Session — who is this user, what session are they in?

2. User Profile — personalization context (role, preferences, language)
 3. Execution Context — what version of the app, what environment?
 4. Product Usage — what did the user do?
 5. Performance — how fast did it load and respond?
 6. Quality / Support — what went wrong and how do we reproduce it?
 7. Privacy and Consent — does the user consent to this collection?
- The "minimum useful" principle: each field must answer a specific question or it should not be collected
 - Anti-pattern: "log everything" — generates noise, increases cost and privacy risk, and nobody reads it

Transitions: This overview gives you the map. Lessons 02.1 and 02.2 walk through each category in detail — what to collect, what it enables, and what to avoid.

Key Principles:

- Every field you collect must enable a specific decision — if you can't state that decision, don't collect the field
- Categories exist to keep data organized and analyzable; unstructured telemetry is unusable telemetry
- Over-collection is not safe; it creates compliance risk, storage cost, and analytical noise

Examples:

- A well-designed event: `{ event: "checkout_completed", userId: "abc123", sessionId: "xyz", route: "/checkout/step-4", timestamp: "2024-01-15T10:30:00Z" }` — every field answers a question
- An over-collected event: `{ event: "click", element: "button", rawHTML: "...", allFormFields: {...} }` — contains PII, is not reproducible without the DOM, and tells you nothing actionable

02.1 Identity, Profile, and Context

Learning Objectives:

- Understand what to collect for user identity, user profile, and execution context
- Distinguish between data that changes per-session and data that changes per-user-update
- Recognize what must NOT go into shared telemetry (PII in identity fields)

Content Outline:

- **Category 1 — Identity / Session:**
 - What: `userId` (hashed/opaque), `sessionId`, `anonymousId` (for pre-login)
 - Why: correlates events across a session and connects events to a user account for support
 - What to avoid: real email addresses, phone numbers, full names — use opaque IDs
- **Category 2 — User Profile (Personalization Context):**
 - What: role/persona, plan/tier, language, timezone, country, UI preferences (theme, notifications), accessibility settings

- Why: enables personalization (content/UX by role/language) and cohort analysis (plan-tier retention)
- Update pattern: these are persistent state values, sent with every event as context — not logged as change events
- **Category 3 — Execution Context:**
 - What: appVersion, environment (dev/staging/prod), current route, referrer, UTM parameters, active feature flags, tenant/academyId
 - Why: without version and environment, a bug report is unreproducible; feature flags identify which variant caused an issue
 - Practical use: "bug only appears on version 2.3.1 with flag `new-checkout` enabled in the EU region"

Transitions: These three categories tell you WHO the user is and WHAT context they're in. Lesson 02.2 covers what they DID, how fast the app responded, and what went wrong.

Key Principles:

- Identity data uses opaque IDs only — email and phone numbers are PII and must never appear in telemetry events
- User profile fields are context attached to every event, not change logs — they tell you what the user's state was when the event occurred
- Execution context is what makes bugs reproducible — without appVersion and feature flags, you cannot reliably reproduce an incident

Examples:

- Good identity: `{ userId: "u_8f3a2b", sessionId: "s_9c4d1e", anonymousId: null }`
- Bad identity: `{ userId: "alice@company.com" }` — this is PII and must not appear in telemetry
- Context saving the day: a user reports a crash — telemetry shows it only happens with `appVersion: "2.4.0", featureFlag: "new-router": true`, on Safari iOS — the fix is immediate because the context was captured

02.2 Behavioral, Performance, and Quality Data

Learning Objectives:

- Understand what to collect for product usage, performance, quality/support, and privacy/consent
- Design meaningful behavioral events using the "what/where/why" framework
- Understand why consent gating is a category of telemetry, not an afterthought

Content Outline:

- **Category 4 — Product Usage (Behavioral Telemetry):**
 - What: navigation events (screen_viewed), key action events (cta_clicked, form_submitted, search_performed, checkout_completed), funnel completion, time-on-step, error interactions, internal search queries

- Naming convention: `subject_verb` pattern (screen_viewed, cta_clicked, form_submitted) — consistent and queryable
- Why: reveals the real user journey, drop-off points, feature adoption, and activation funnels
- Anti-pattern: `{ event: "click" }` — tells you nothing; `{ event: "cta_clicked", location: "pricing_page", variant: "A" }` tells you everything
- **Category 5 — Performance:**
 - What: Web Vitals (LCP, CLS, FID/INP), API call latency (per endpoint), client-side render times, API error rates
 - Why: performance degradation is invisible without measurement — a 200ms regression on the checkout API directly impacts conversion
 - Collection method: `reportWebVitals` in Next.js, fetch/axios interceptors for API latency
- **Category 6 — Quality / Support:**
 - What: errorId, error fingerprint, sanitized stack trace, breadcrumbs (last N navigation events before the error), networkState, apiStatus
 - Why: reproducibility — with this data, a support ticket turns into a reproducible bug in minutes
 - Sanitization: stack traces must be scrubbed of user input and PII before transmission
- **Category 7 — Privacy and Consent:**
 - What: `consent.analytics`, `consent.marketing`, `consent.functional` — boolean flags per consent type
 - Why: consent gating is a legal requirement (GDPR, CCPA) and an ethical baseline — analytics must not fire if `consent.analytics` is false
 - Implementation: a single consent gate at the telemetry layer, not scattered across individual event calls

Transitions: You now have the full picture of what to collect and why. The next section addresses a fundamental structural question: where does user profile data live, and how is it different from usage event data?

Key Principles:

- Behavioral events must answer three questions: what happened, where it happened, and why it was triggered (which feature, which variant)
- Performance data is not optional — you cannot improve what you don't measure
- Consent gating is binary: if consent is not given, the event must not fire, regardless of how useful the data would be
- Error data without context is useless; error data WITH session + context + breadcrumbs is immediately actionable

Examples:

- Good behavioral event: `{ event: "search_performed", query_length: 12, results_count: 0, route: "/courses" }` — notice the query LENGTH not the query string (PII risk)
- Performance alert: API telemetry shows `/api/enroll` P95 latency jumping from 180ms to 1.2s after a deploy — caught before users file complaints

- Anti-pattern: `{ event: "form_submitted", formData: { email: "user@example.com", comment: "..." } }` — raw PII in an event is a compliance violation
-

03.0 User Profile vs Usage Data

Learning Objectives:

- Explain the fundamental structural difference between user profile data and usage event data
- Apply the correct storage and transmission pattern for each type
- Avoid the anti-pattern of mixing profile state and behavioral events

Content Outline:

- The core distinction:
 - **User Profile** = persistent state about who the user IS — role, plan, language, preferences
 - **Usage Data** = append-only record of what the user DID — events, actions, interactions
- How each is stored and updated:
 - User profile: stored as state (database record), updated via `PATCH /users/me` when the user changes a setting; attached as context to every telemetry event, not logged as individual events
 - Usage data: sent as individual events to `POST /telemetry/events`, never updated — each event is a permanent record of something that happened
- Why the distinction matters:
 - Mixing them creates analytics problems: if you log a "profile updated" event every time you read profile data, your funnel analysis is corrupted
 - Profile data is context; event data is the record
 - Querying "what did this user do" and "who is this user" are fundamentally different queries, and mixing the data makes both worse
- The event envelope: the standard wrapper for every usage event: `{ event, timestamp, user: {...profileContext}, session: {...}, context: {...executionContext}, properties: {...eventSpecificData}, schemaVersion }`

Transitions: You now understand what to collect, why, and how to structure it. The assessment in section 04 will test your ability to apply these decisions to realistic scenarios.

Key Principles:

- User profile = state (who they are now); usage data = log (what they did)
- Profile data is attached as context to events; it is never the event itself
- Append-only event logs are the backbone of product analytics — they must not be corrupted with profile state changes

Examples:

- Correct: When a user changes their language to French → update `users.language = 'fr'` in the database; the next telemetry event will automatically carry `{ user: { language: 'fr' } }` as context

- Incorrect: When a user changes their language → `log { event: "language_changed", newLanguage: "fr", email: "user@example.com" }` — this mixes state change with event logging and includes PII
 - The event envelope in practice: `{ event: "checkout_completed", timestamp: "2024-01-15T10:30:00Z", user: { userId: "u_8f3a2b", role: "student", plan: "pro" }, session: { sessionId: "s_9c4d1e" }, context: { appVersion: "2.4.0", route: "/checkout/step-4" }, properties: { courseId: "c_123", amount: 49.99 }, schemaVersion: "1.0" }`
-

04.0 Frontend Telemetry Assessment 🧐

Learning Objectives:

- Apply the data category framework to realistic product scenarios
- Identify what to collect, why, and which category each piece of data belongs to
- Recognize anti-patterns in telemetry design

Question Format: Scenario-based

Questions:

1. A product team wants to understand why users abandon the registration flow. Which telemetry category is most directly relevant, and what specific events should be collected at each step?
2. A developer receives a bug report: "The app crashes sometimes." With no telemetry, this is unresolvable. Which two categories of telemetry data would make this reproducible, and what specific fields from each category are essential?
3. Your Next.js app serves users in France and Germany. Under GDPR, you must not collect analytics data without explicit consent. Which telemetry category handles this, and what is the correct implementation pattern?
4. A user changes their notification preferences in the app settings. Should this be recorded as a telemetry event, a user profile update, or both? Explain the correct approach and why mixing them is an anti-pattern.
5. An engineer proposes the following event: `{ event: "form_submit", formData: { name: "Alice Smith", email: "alice@example.com", message: "I can't log in" } }`. Identify two problems with this event design and explain how to fix them.
6. Your API serves 50,000 requests per day. The P95 latency on `/api/search` doubled after last week's deploy, but no users have complained yet. Which telemetry category caught this, and why is "no complaints = no problem" a dangerous assumption?
7. A startup's telemetry event is: `{ event: "click" }`. This event fires 200,000 times per day but generates zero actionable insights. Rewrite it as a properly structured event for a "Sign Up" button on the pricing page, including the correct fields from the behavioral telemetry category.

8. You are building a telemetry system and decide to include the user's email address as the primary identifier in every event, arguing it makes support easier. Which category explicitly prohibits this, and what should you use instead?

Evaluation Criteria:

- Correctly maps scenarios to the appropriate telemetry category
- Identifies PII risks and proposes privacy-safe alternatives
- Distinguishes user profile from usage event data
- Applies the "what decision does this enable?" test to evaluate event design

Score Interpretation:

- 7-8 correct: Ready to design a telemetry system for a production application
 - 5-6 correct: Solid foundation; review data categories and the profile vs. events distinction
 - Below 5: Re-read sections 02 and 03 before implementing any telemetry
-

05.0 Conclusion

Content Outline:

- Course recap: what students accomplished
 - Understood what frontend telemetry is and why it is essential for modern product development
 - Learned the seven categories of data worth collecting: identity, user profile, execution context, product usage, performance, quality/support, and privacy/consent
 - Applied the "minimum useful set" principle: every field must enable a specific decision
 - Mastered the fundamental distinction between user profile data (persistent state) and usage event data (append-only log)
 - Understood the standard event envelope that structures every telemetry event
- Connection to bigger picture
 - This course covered what to collect — the next skill (Skill 44) covers how to send it reliably from the frontend: batching, debouncing, sendBeacon on page unload, retry queues, and correlation IDs
 - The telemetry data you design here feeds directly into reports, dashboards, and ML training pipelines in later skills
 - The privacy and consent patterns you learned are not optional — they are the foundation for GDPR and CCPA compliance in production systems
- Next steps and continued learning
 - Explore: Google's Web Vitals documentation for the performance category
 - Explore: the GDPR consent requirements for analytics (specifically Article 6 and 7)
 - Read: "Amplitude Data Taxonomy" — a real-world example of how companies structure behavioral event naming
 - Practice: open an application you use daily and reverse-engineer what its telemetry events probably look like based on the features you see
- Encouragement

- Telemetry design looks simple but it is one of the highest-leverage skills in product development — the data you decide to collect today will determine what decisions your team can make six months from now. You now have the framework to make those decisions intentionally.

Note: This is conclusion only — no theory, exercises, or assessment.